

IES: AgriSense - Product specification report

Diego Aguilar [117122], Gonçalo Fonseca [118920], Marta Condeço [112903], Tiago Coelho [118745]

v2026-03-18

1.1 Overview of the project	1
1.2 Project limitation	2
2.1 Vision statement	2
2.2 Personas and scenarios	2
3.1 Key quality requirements	3
Scalability	3
Reliability & Availability	3
Usability	3
3.2 Architecture view	4

1 Introduction

1.1 Overview of the project

This project was developed within the scope of the Introduction to Software Engineering curricular unit and focuses on the creation of a modern, user-centric dashboard platform for farm management.

The platform is designed to provide **fast, centralized access to multiple dashboards**, each presenting critical information about the status and performance of farm animals.

The system aggregates key metrics, such as:

- Health indicators
- Activity levels
- Environmental conditions

This enables users to monitor their livestock efficiently and make informed decisions in real time.

With an emphasis on clarity, responsiveness, and ease of use, the platform aims to simplify complex data into intuitive visual insights, ultimately supporting better farm management and improving overall operational awareness.

1.2 Project limitation

As this project is a proof of concept and does not incorporate real actuators, it operates within a simulated environment rather than relying on real-world data. While the simulation is designed to closely mimic realistic conditions, the results may not fully reflect the behavior and variability of a real-world deployment.

2 Product concept and requirements

2.1 Vision statement

The system addresses the growing need for advanced animal tracking solutions that enable farmers to monitor livestock in real time and respond rapidly to potential issues. It provides a comprehensive platform for managing multiple barns within a farm, each housing different animal species.

Within each barn, the system continuously monitors individual animals by collecting key health and activity metrics, including body temperature, movement, heart rate, location, respiratory rate, and posture (e.g., standing or resting). In addition, environmental conditions are tracked to provide a complete view of the animals' surroundings.

By combining real-time data collection with intuitive visualization, the platform empowers farmers to make informed decisions, improve animal welfare, and enhance overall farm efficiency.

2.2 Personas and scenarios

Persona 1: Farmer

Motivation: To easily monitor the vital signs and overall well-being of livestock in real time, enabling faster and more informed decision-making.

Scenario:

"It's a Sunday morning after several particularly cold days, and I'm concerned about the well-being of my animals, even though I check on them daily. I want a quick and reliable way to assess their condition without having to inspect each one manually.

I open the AgriSense app and immediately access the dashboard overview. I notice that one of my animals is showing abnormal vital signs. Recognizing the urgency, I quickly contact my veterinarian through the app, which automatically shares the animal's current status and relevant data."

3 Architecture notebook

3.1 Key quality requirements

Performance

The system must provide fast and responsive access to dashboards, allowing users to view animal data with minimal delay (maximum 5 seconds). Since real-time monitoring is critical, the platform should support low-latency data updates and efficiently handle frequent incoming data streams from multiple animals. Dashboard loading times should remain short, even as data volume increases.

Scalability

The platform should be able to scale to support multiple farms, each with several barns and a large number of animals. As the number of monitored metrics and incoming data messages grows, the system must handle increased load without performance degradation.

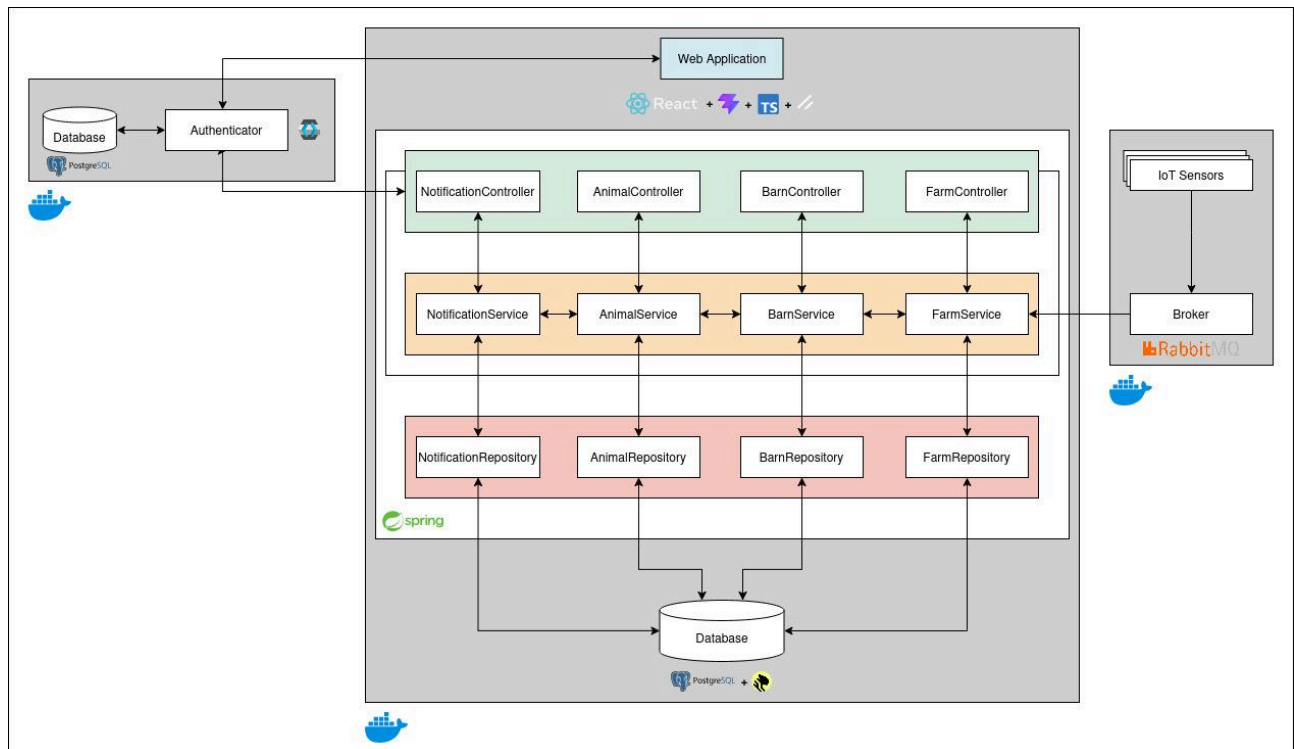
Reliability & Availability

The system must ensure continuous operation and at least 99% availability, as farmers rely on it for timely insights into animal health. It should be resilient to failures, with mechanisms for fault tolerance, message recovery, and data consistency. In case of component failure, the system should recover quickly without significant data loss.

Usability

The platform should provide an intuitive and user-friendly interface, enabling farmers to quickly understand the status of their animals. Dashboards must present complex data in a clear and accessible way, using visualizations and alerts to highlight critical situations. The system should minimize the time and effort required to interpret data and take action.

3.2 Architecture view



3.2.1 Technology Stack

For the frontend we will be using **React**, **Vite**, **TypeScript**, and for components, we will use **ShadCN**, **Lucide**, and **Recharts**.

To authenticate our users and manage their permission we will use **KeyCloak**.

For the backend, we will use **Spring** and structure our project by using **Maven** with no archetype.

Regarding the database, we will be using a **PostgreSQL** with **TimescaleDB** extension.

To handle all the incoming IoT sensor data, we will be using a message broker, specifically, **RabbitMQ**.

In the end, all of this will be containerized by the use of **Docker**.

4 Information Model

4.1 Swagger

Animals CRUD operations for animals			^
POST	/api/animals		▼
GET	/api/animals/{id}/weight	Get animal weight	▼
GET	/api/animals/{id}/movement	Get animal movement	▼
GET	/api/animals/{id}/metrics/latest	Get latest metrics	▼
GET	/api/animals/barn/{barnId}	Get barn	▼
DELETE	/api/animals/{id}	Delete animal	▼

Farms CRUD operations for farms			^
PUT	/api/farms/{id}	Update farm	▼
POST	/api/farms		▼
GET	/api/farms/farmer/{farmerId}	Get farmer's farms	▼

Barns CRUD operations for barns			^
GET	/api/barns/{id}	Get barn by ID	▼
PUT	/api/barns/{id}	Update a barn	▼
DELETE	/api/barns/{id}	Delete a barn	▼
GET	/api/barns	Get all barns	▼
POST	/api/barns	Create a barn	▼

Manager CRUD operations for manager			^
PUT	/api/manager/farmers/{id}	Update a farmer	▼
DELETE	/api/manager/farmers/{id}	Delete a farmer	▼
GET	/api/manager/farmers	Get farmers list	▼
POST	/api/manager/farmers	Create farmer	▼